Argo

Manual

Instalação, telas, arquitetura, API, operação e solução de problemas

Copyright (©) 2026 Erik Pimenta

Versão 1.2.0 · Novembro de 2026 · Licença MIT

Sumário

- **Visão geral** 4
  - O que a plataforma faz 4
  - Serviços 4
- **Como rodar** 6
  - Primeira execução 6
  - Comandos do dia a dia 6
  - Contas de demonstração 7
  - Sem Docker 7
- **Acesso e segurança** 8
  - Regras 8
- **Guia das telas** 10
  - Loja e carrinho 10
  - Pedidos 11
  - Produtos e estoque 12
  - Notas fiscais 14
  - Usuários e Sistema 14
  - Minha conta, tema e paletas 16
  - Busca rápida e atalhos 17
- **Arquitetura** 19
  - Como o Django está organizado 19
  - Do clique à nota fiscal 20
- **Processamento assíncrono** 21
- **API** 22
- **Configuração** 24
- **Testes e automação** 25
  - Integração contínua 25
- **Operação e manutenção** 26
  - Diagnóstico 26
  - Backup 26
  - Atualização 26
  - Logs 26
  - Administrador de emergência 26
- **Solução de problemas** 27

- **Novidades da 1.2.0** 28
  - Como atualizar 28
  - Versão 1.1.0 28

Capítulo 1

Visão geral

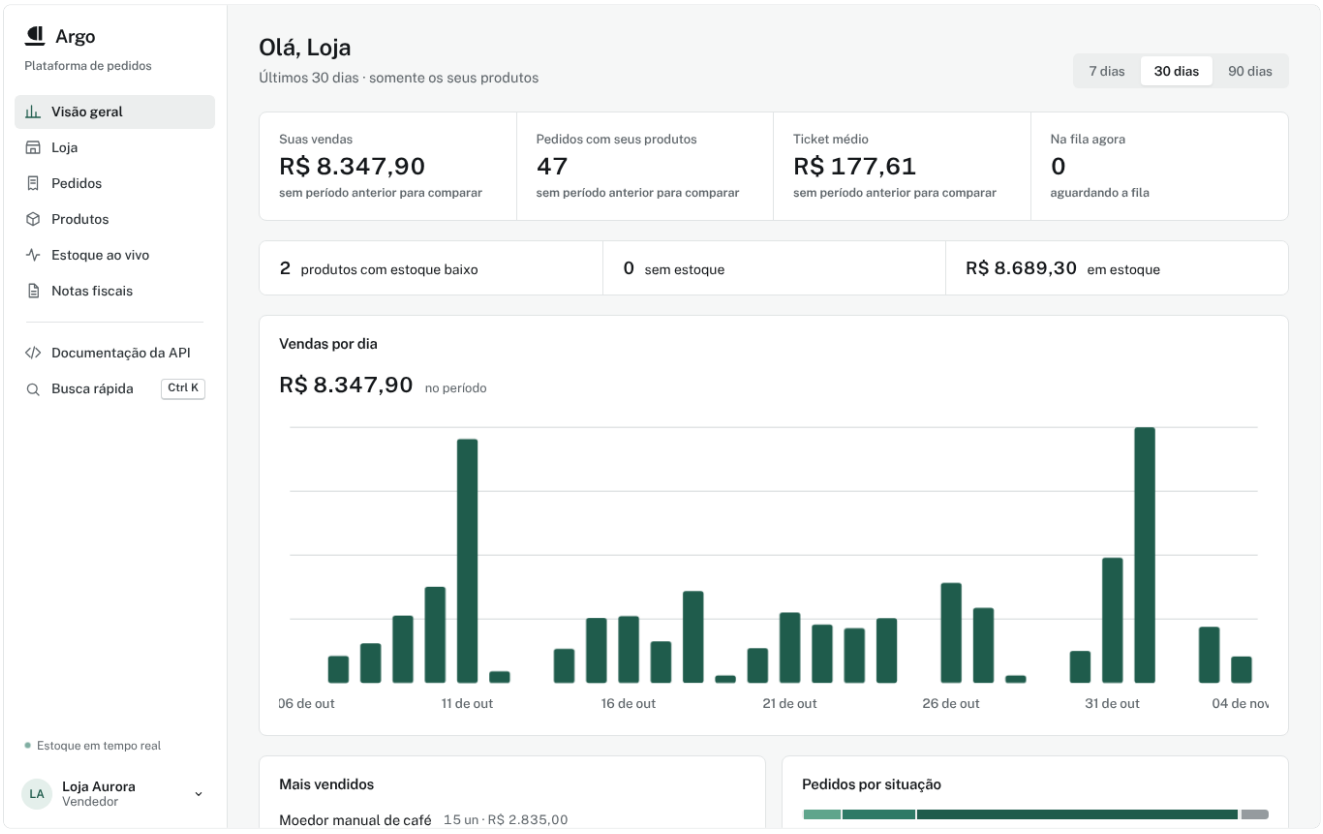
O Argo é uma plataforma de microserviços para vendas: vários vendedores cadastram produtos, o estoque muda em tempo real em todas as telas e cada pedido é confirmado por uma fila, sem travar a compra.

O que a plataforma faz

- **Cadastro e papéis:** qualquer pessoa cria conta de Cliente ou Vendedor. O Administrador cria outros administradores, troca papéis, redefine senhas, desbloqueia e bloqueia contas.
- **Catálogo multivendedor:** cada vendedor cuida dos próprios produtos, com preço, categoria, estoque mínimo e histórico de toda movimentação.
- **Estoque em tempo real:** toda alteração é publicada no Redis e chega na hora a todas as telas abertas por Server-Sent Events.
- **Checkout assíncrono e idempotente:** o pedido responde `202 Accepted` em milissegundos e o cabeçalho `Idempotency-Key` evita pedido em dobro. Estoque, e-mails e nota fiscal acontecem na fila do Celery.
- **Faturamento em Java:** um serviço Spring Boot emite e cancela as notas, com o valor aproximado dos tributos.
- **Painel por papel:** faturamento, ticket médio, vendas por dia, mais vendidos e comparação com o período anterior, cada um vendo só o que lhe cabe.
- **Exportação:** pedidos, produtos e notas saem em CSV pronto para o Excel, com os mesmos filtros da tela.
- **Operação simples:** `make` no Linux e macOS e `argo` no Windows sobem, param, testam, diagnosticam, fazem backup e restauram a plataforma.

Serviços

Serviço	Linguagem	Responsabilidade
`identity`	Python · Django · DRF	Cadastro, login, JWT, refresh token com rotação, sessões e gestão de usuários
`catalog`	Python · Django · DRF	Produtos, estoque com trava de linha, histórico, exportação e eventos em tempo real
`orders`	Python · Django · DRF	Checkout idempotente, situação dos pedidos, cancelamento, indicadores e exportação
`worker`	Python · Celery	Baixa no estoque, e-mails, nota fiscal, devoluções e recuperação
`scheduler`	Python · Celery beat	Recuperação de pedidos parados e limpeza de sessões vencidas
`billing`	Java 21 · Spring Boot 4	Emissão, cancelamento, busca e exportação das notas fiscais
`gateway`	Nginx	Interface web, documentação, roteamento, limites e cabeçalhos de segurança



Visão geral do vendedor

Capítulo 2

Como rodar

Você só precisa do Docker com Docker Compose. Os scripts criam o arquivo `.env` com senhas e segredos aleatórios na primeira vez.

Primeira execução

Sistema	Comando
Windows	Dois cliques em `iniciar.cmd`
Linux ou macOS	`./iniciar.sh`
Sem script	`cp .env.example .env` e `docker compose up -d --build`

Os 11 contêineres sobem e o navegador abre em http://localhost:8080. A primeira execução demora alguns minutos para baixar as imagens; as próximas sobem em segundos.

Endereço	O que é
http://localhost:8080	Aplicação
http://localhost:8080/docs/	Documentação da API (Swagger) dos quatro serviços
http://localhost:8025	Caixa de entrada dos e-mails de teste (Mailpit)
http://localhost:5555	Monitor das filas (Flower)

Comandos do dia a dia

No Linux e no macOS use `make <comando>`; no Windows, `argo <comando>` (o `argo.cmd` chama `scripts/argo.ps1`). `make ajuda` lista tudo.

Comando	O que faz
`subir` · `parar` · `reiniciar`	Sobe com build, para e reinicia os contêineres
`logs` · `situacao` · `saude`	Acompanha os logs, mostra os contêineres e consulta o `/health` de cada serviço
`diagnostico`	Roda o `doctor`: bancos, migrações, Redis, SMTP, faturamento e segredos
`admin`	Cria ou atualiza um administrador (`EMAIL=` e `SENHA=` ou perguntando)
`recuperar`	Põe de novo na fila os pedidos parados
`limpar-sessoes`	Apaga refresh tokens vencidos
`backup` · `restaurar`	Copia e restaura os quatro bancos em `backups/`
`testes` · `testes-java` · `lint`	Testes Python, testes Java e verificação de estilo
`esquema`	Regenera `database/*.sql` e `docs/banco-de-dados.md`

Comando	O que faz
`atualizar`	`git pull`, backup e nova subida com as migrações
`zerar`	Remove contêineres e volumes (apaga os dados, pede confirmação)

Contas de demonstração

Todas usam a senha `Argo@2026`. Existem só para testes e somem se a plataforma subir com `ARGO_SEED=false` em bancos vazios.

Papel	E-mail	O que pode fazer
Administrador	`admin@argo.dev`	Tudo, incluindo usuários e pedidos de qualquer vendedor
Vendedor	`vendedor@argo.dev`	Produtos, estoque, pedidos com seus produtos e suas notas
Cliente	`cliente@argo.dev`	Loja, carrinho, pedidos e notas fiscais próprias

Para produção, defina `ARGO_ADMIN_EMAIL` e `ARGO_ADMIN_PASSWORD`: o primeiro administrador é criado na subida, mesmo sem dados de demonstração.

Sem Docker

```
cd backend
pip install -r requirements-dev.txt
ARGO_SERVICE=all python manage.py serve
ARGO_SERVICE=worker celery -A argo.worker.celery_app:celery worker -Q orders
```

Com `ARGO_INLINE_QUEUE=true` as tarefas rodam no próprio processo, sem Redis nem worker, o que ajuda a depurar o fluxo inteiro num só terminal.

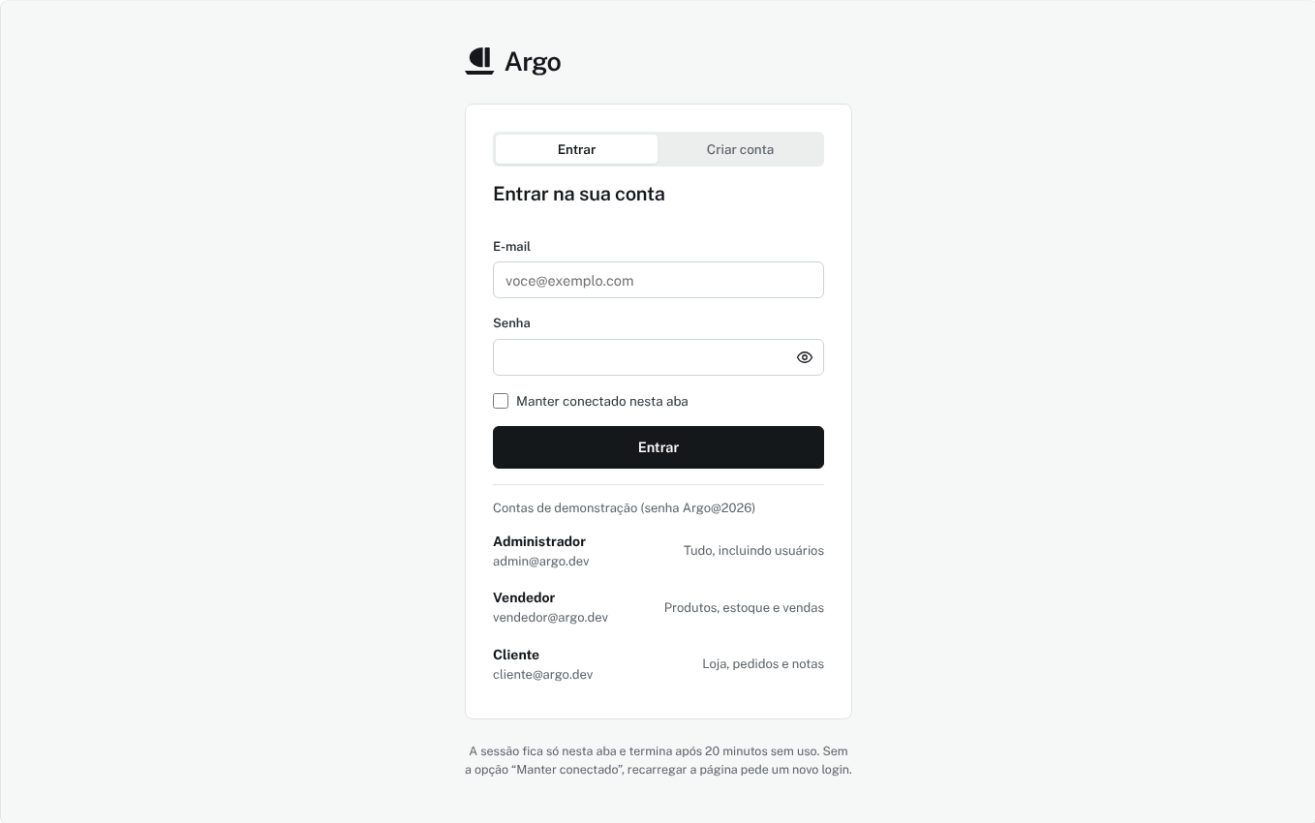
Capítulo 3

Acesso e segurança

Toda tela exige login. Os papéis decidem o que cada pessoa vê e altera, inclusive na API.

Regras

Regra	Como funciona
Token de acesso	JWT HS256 de 15 minutos com `sub`, `role`, `name`, `email`, `sid`, `iss`, `exp` e `jti`, validado por todos os serviços, inclusive o Java
Refresh token	Opaco, de uso único e válido por 7 dias. Só o SHA-256 fica no banco. Cada troca gera um novo e invalida o anterior
Sessões	Cada login abre uma sessão com aparelho, IP, início e último uso. Em Minha conta dá para encerrar uma sessão ou todas as outras
Detecção de roubo	Reapresentar um refresh token já trocado encerra a sessão inteira
Manter conectado	Opcional no login: guarda o refresh token só na aba (sessionStorage) e recarregar a página não pede login
RBAC	`ADMIN`, `SELLER` e `CUSTOMER` checados em cada rota
Senhas	bcrypt do Django, mínimo de 8 caracteres com letras e números. Cinco erros seguidos bloqueiam a conta por 5 minutos; o administrador pode desbloquear
Troca de senha	Encerra as outras sessões e mantém a atual. A redefinição pelo administrador encerra todas
Entre serviços	Rotas `/internal/*` exigem `X-Internal-Token`; o gateway bloqueia essas rotas e descarta o cabeçalho vindo de fora
Limites	10 tentativas por minuto por IP em login e cadastro e 30 requisições por segundo por IP no restante da API (HTTP 429)
Cabeçalhos	Content-Security-Policy, Cross-Origin-Opener-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy e Permissions-Policy em todas as respostas
Segredos	Os serviços Python e Java avisam no log, e o `make diagnostico` aponta, quando os segredos são os de exemplo do `.env.example`, curtos ou iguais entre si



Entrada com as contas de demonstração

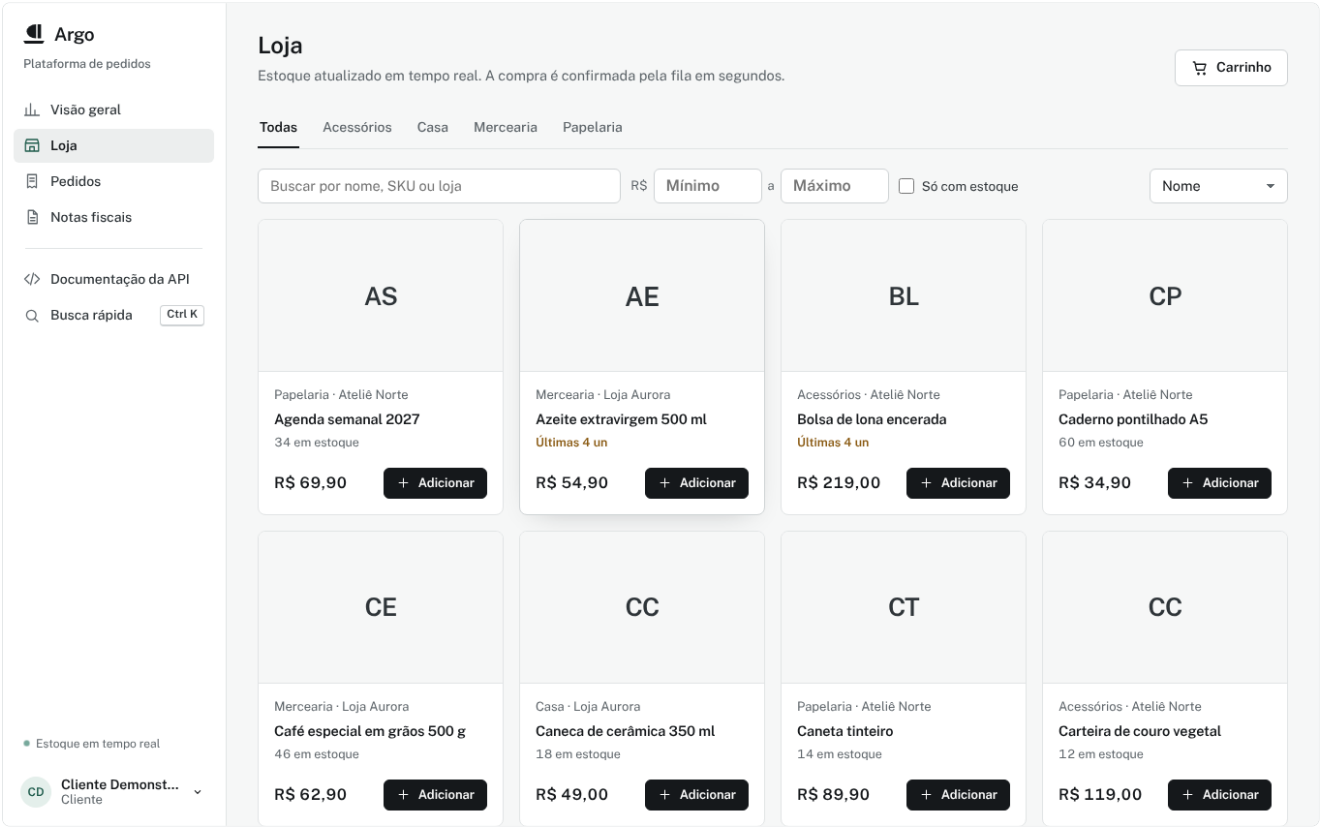
Capítulo 4

Guia das telas

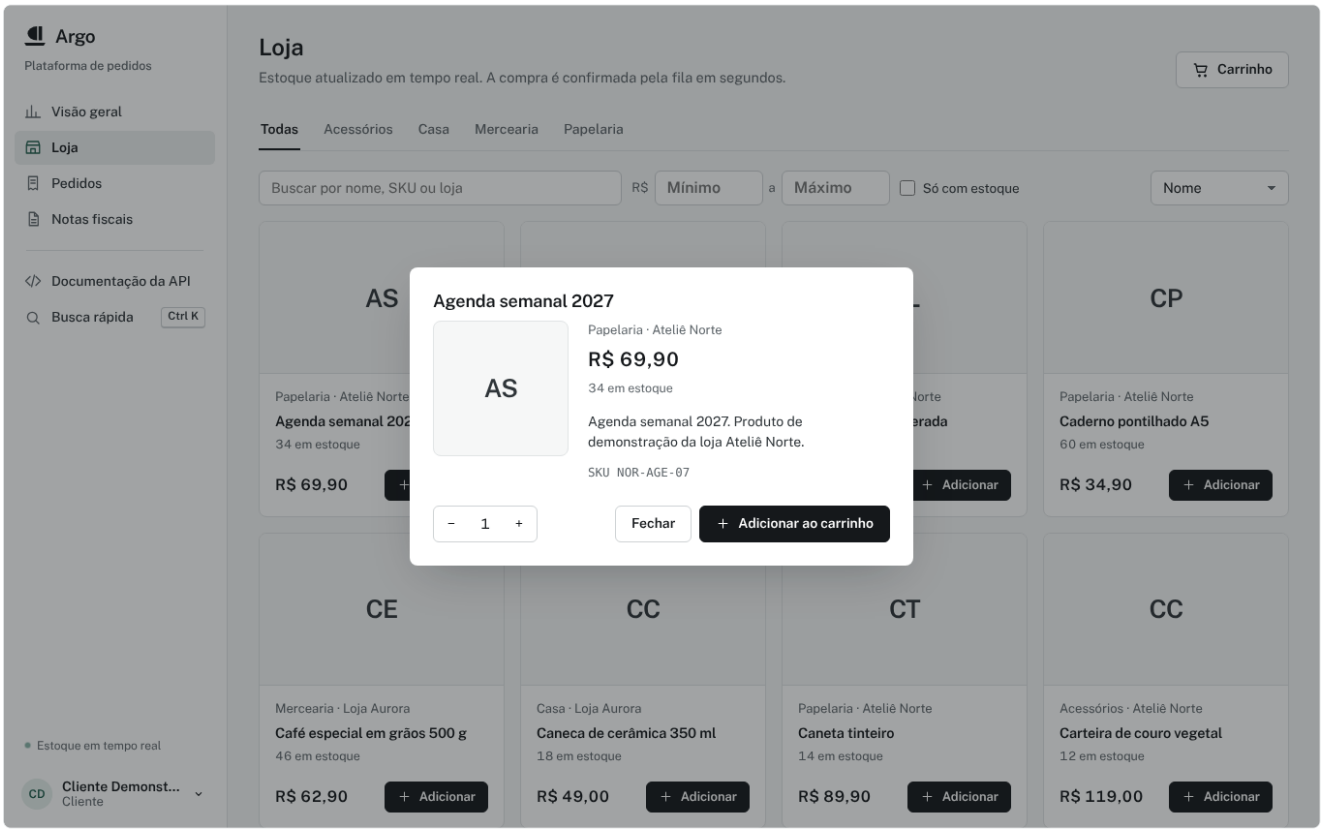
A interface é a mesma para os três papéis; o menu mostra só o que cada um pode usar.

Loja e carrinho

O cliente busca por nome ou SKU, filtra por categoria e vê o estoque mudar sem recarregar. O carrinho fica numa gaveta lateral, guarda os itens entre visitas e avisa se algum produto acabou antes do checkout. Cada produto abre uma página com descrição, estoque e vendedor.



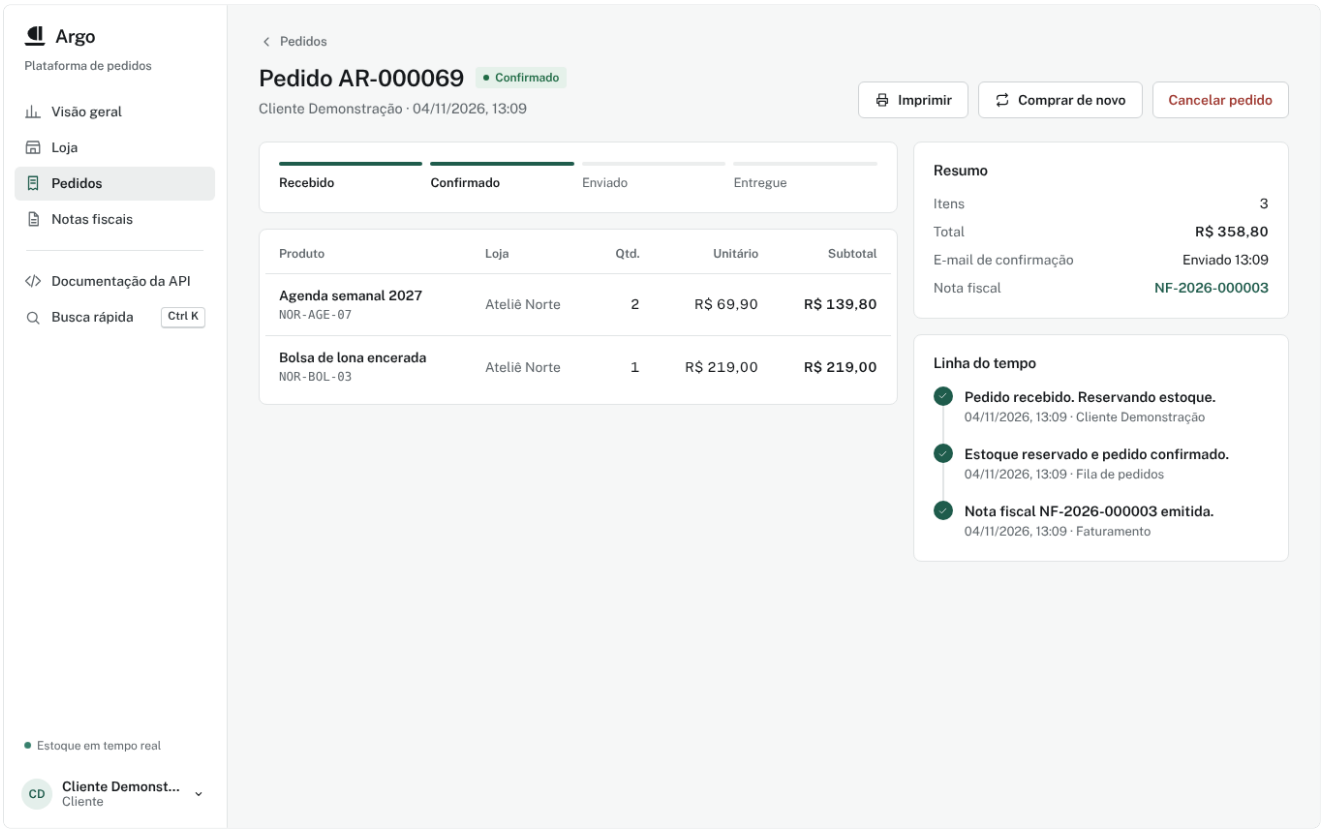
Loja com estoque em tempo real



Página do produto

Pedidos

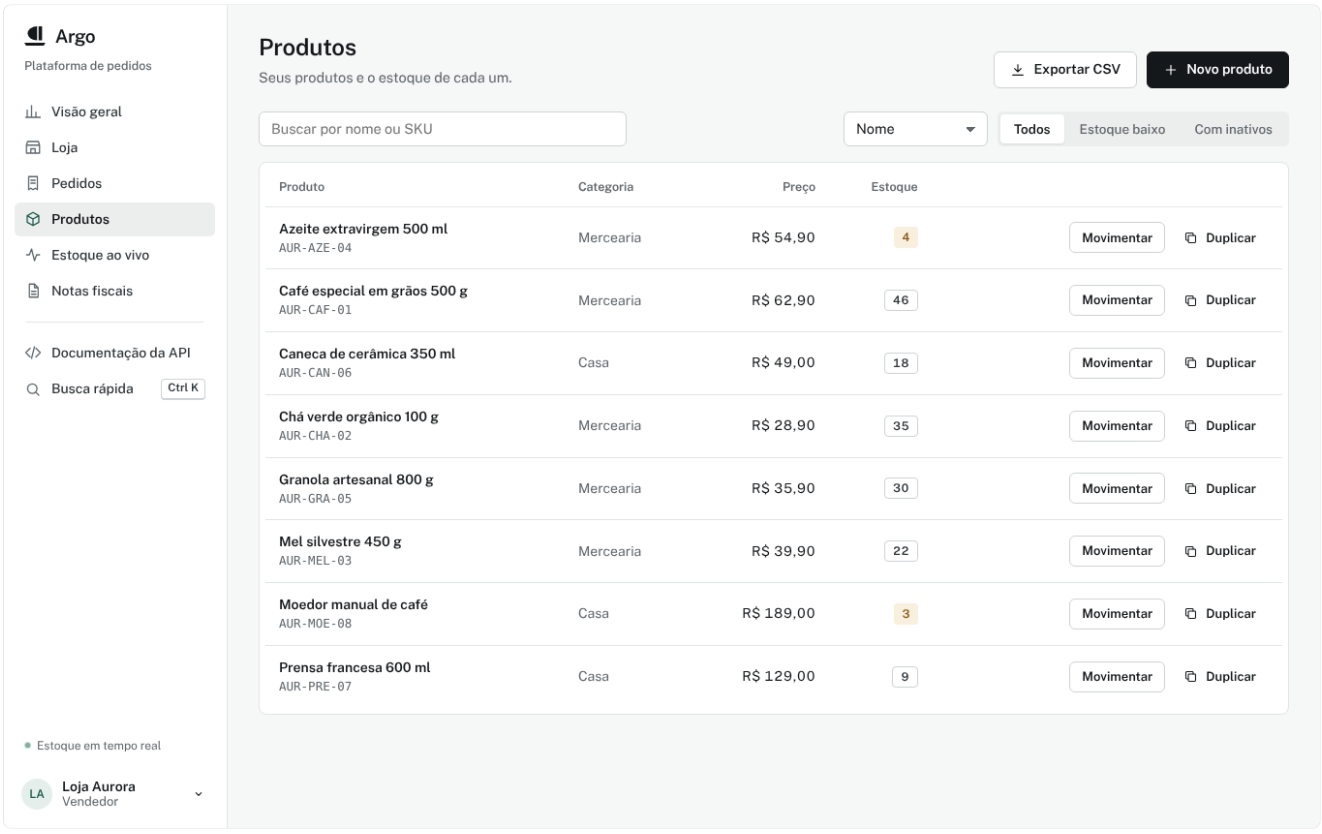
A lista tem busca, filtros por situação e período, paginação e exportação em CSV. A página do pedido mostra a linha do tempo (fila, e-mail, nota, envio e entrega) e se atualiza sozinha enquanto o pedido está em processamento. O vendedor informa o envio com código de rastreio; o cliente recebe um e-mail a cada mudança.



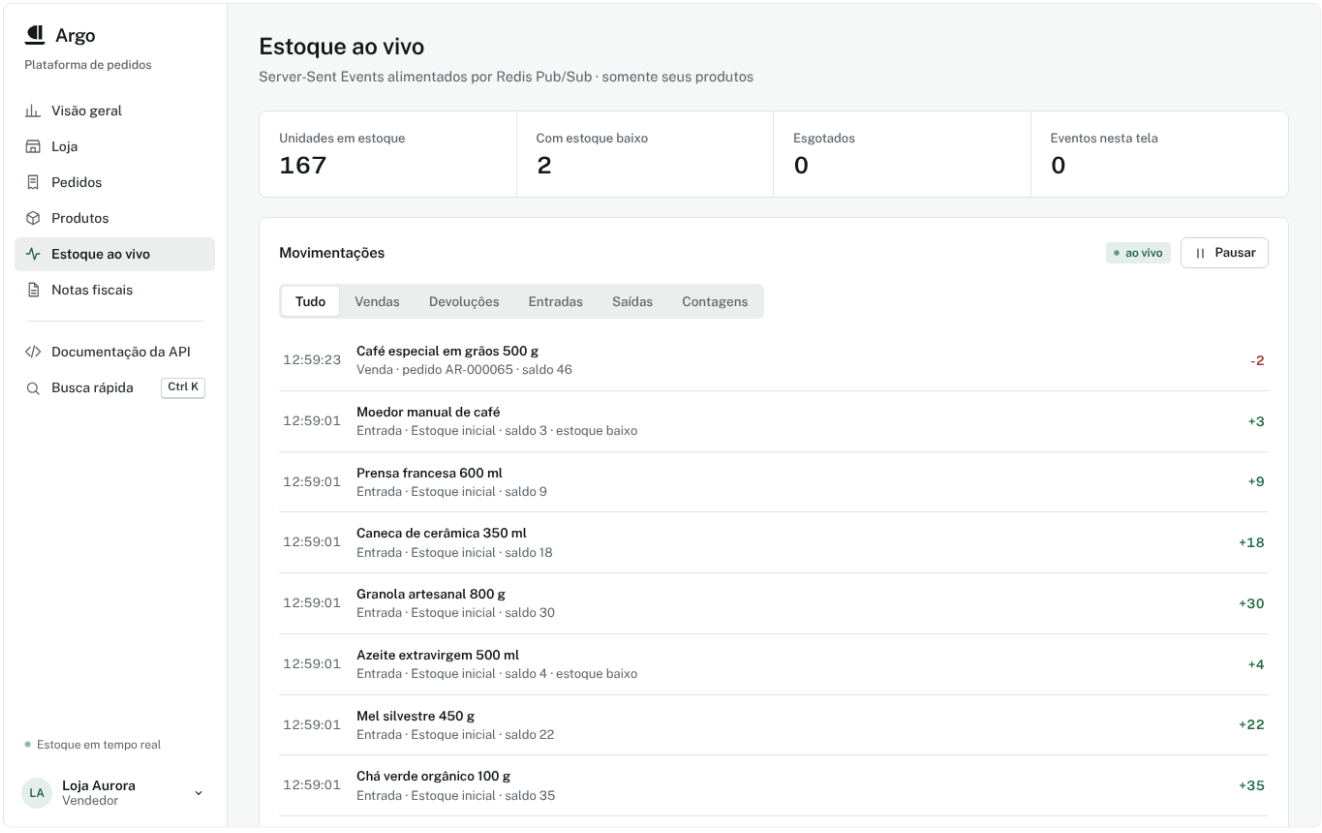
Pedido confirmado pela fila, com e-mail enviado e nota emitida

Produtos e estoque

O vendedor cadastra e edita produtos, ajusta o estoque (entrada, saída ou contagem) e vê o histórico de cada item. A tela Estoque ao vivo junta as movimentações de todos os produtos, com filtro e carregamento de mais itens.



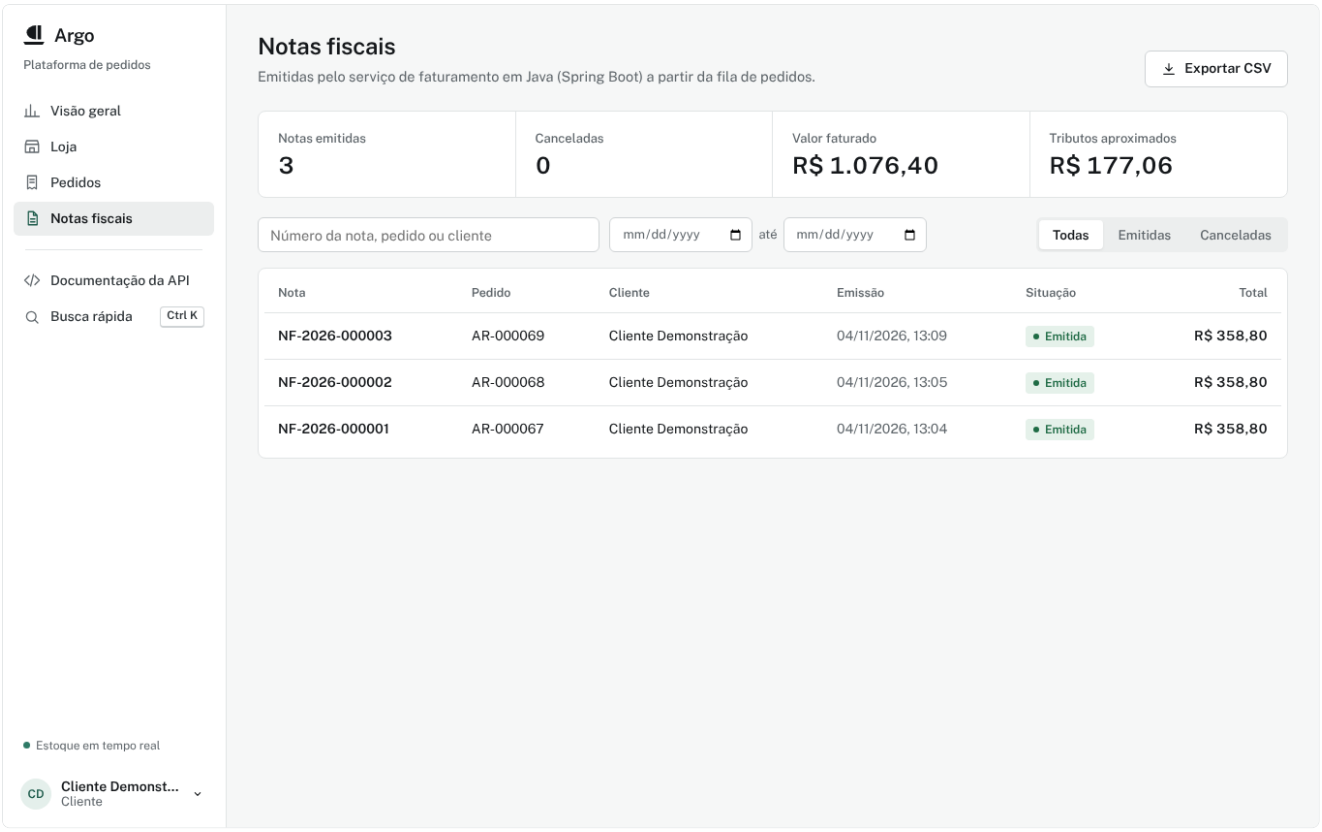
Produtos do vendedor



Movimentações de estoque ao vivo

Notas fiscais

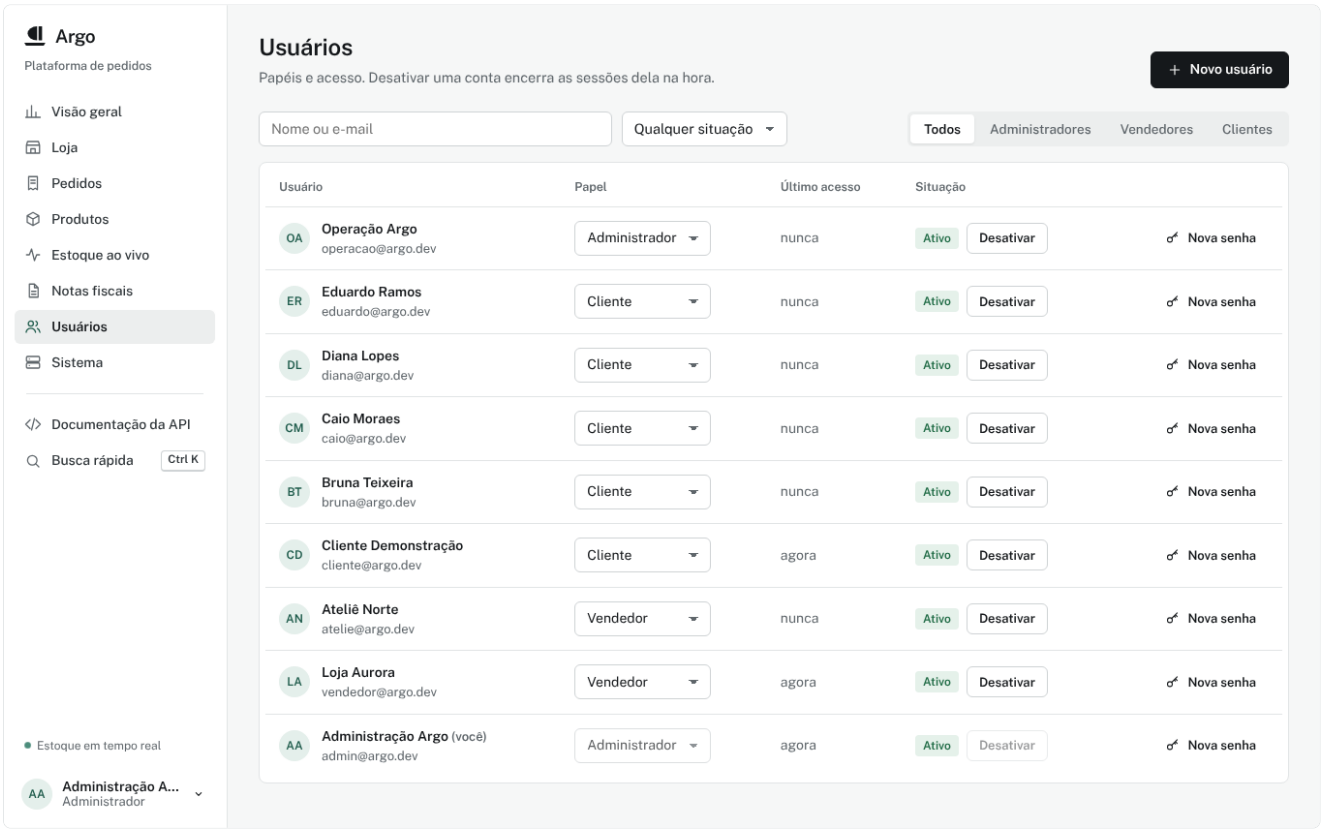
Busca por número, pedido ou cliente, filtros por situação e período, resumo do total e dos tributos e exportação em CSV. Cada nota abre em detalhe e pode ser impressa.



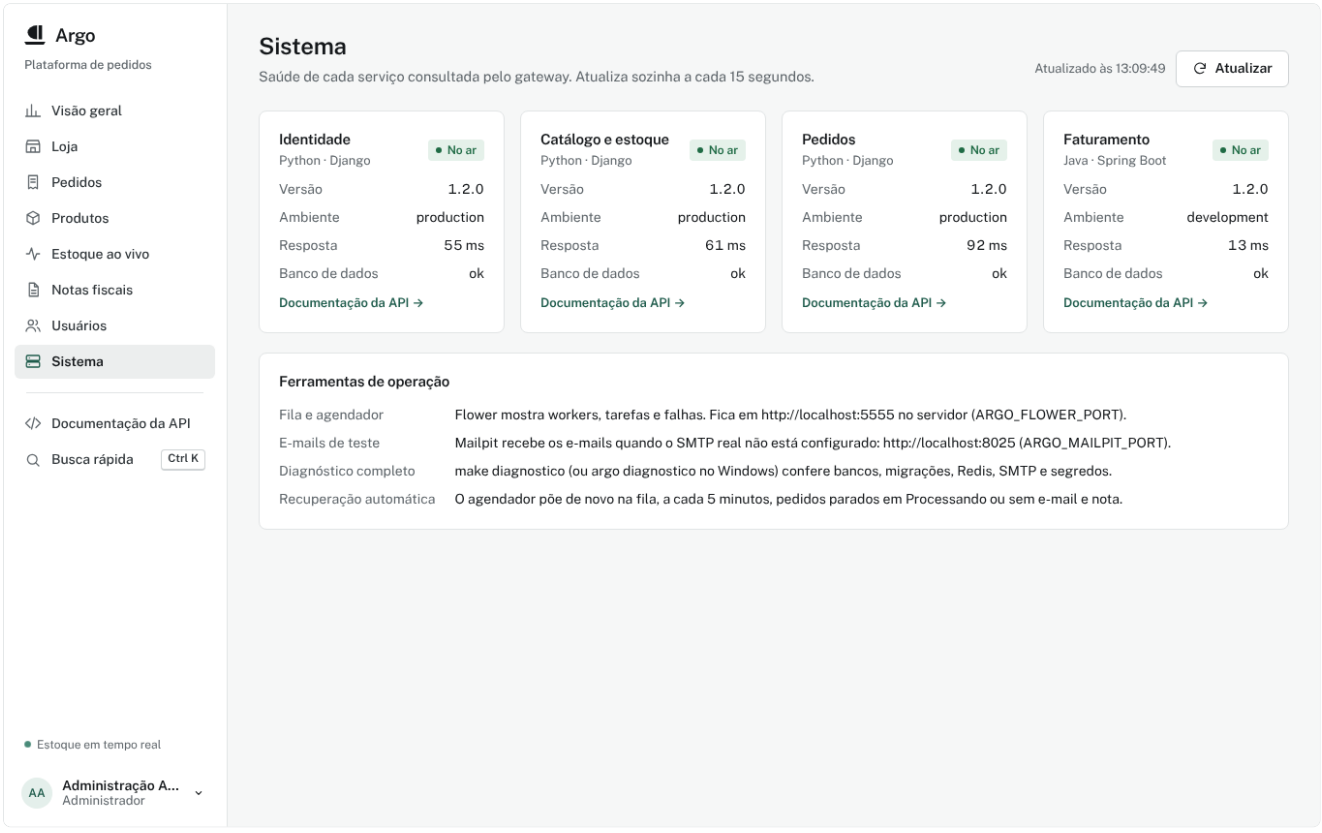
Notas fiscais com filtros e exportação

Usuários e Sistema

O administrador cria contas, troca papéis, bloqueia, desbloqueia e redefine senhas. A tela Sistema mostra a saúde de cada serviço (versão, tempo de resposta e bancos), atualizada a cada 15 segundos.



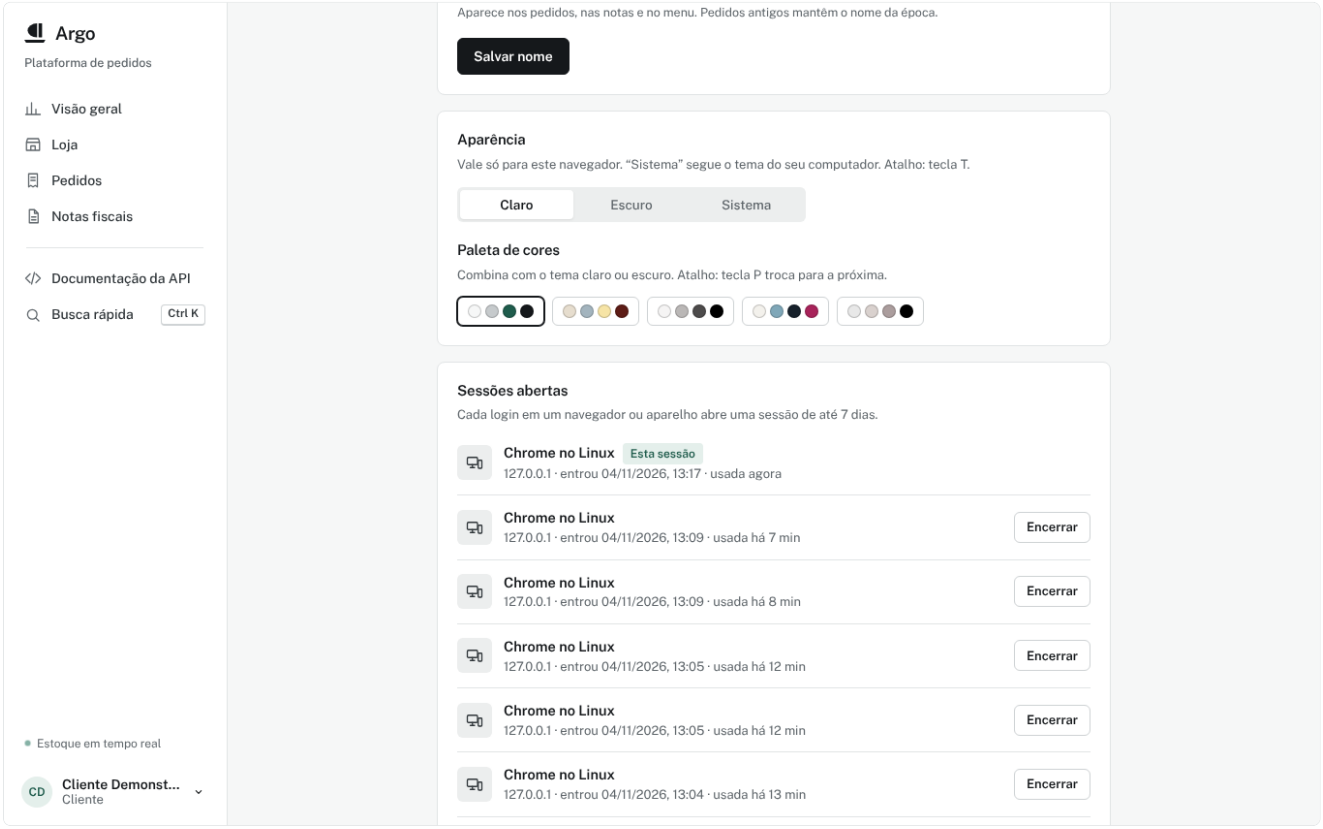
Gestão de usuários



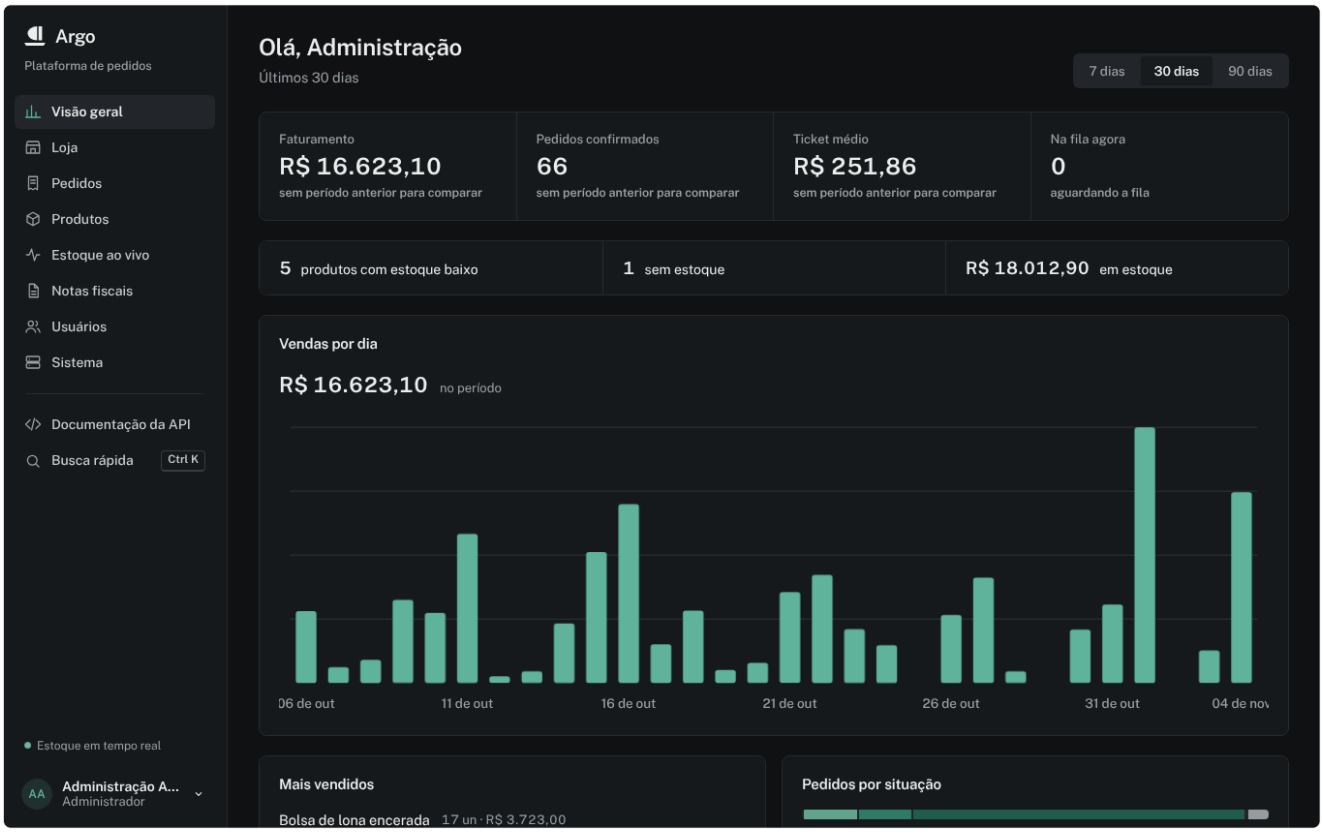
Saúde dos serviços

Minha conta, tema e paletas

Em Minha conta cada pessoa troca o nome e a senha, vê as sessões abertas e escolhe a aparência: tema claro, escuro ou o do sistema e uma entre cinco paletas de cores. A escolha vale para o navegador e também fica no menu do usuário. Atalhos: `T` alterna o tema e `P` passa para a próxima paleta.



Minha conta com tema, paletas e sessões abertas

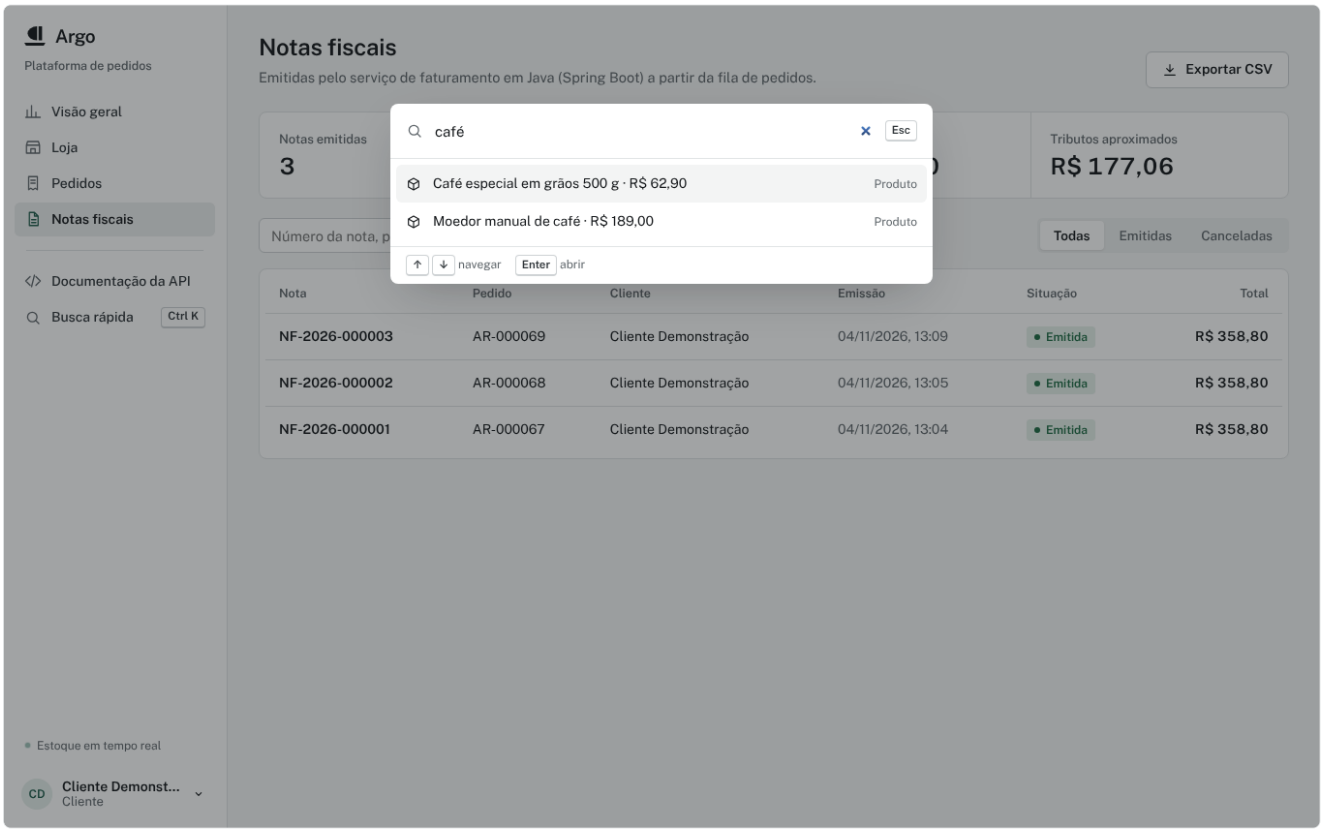


Tema escuro

Busca rápida e atalhos

`Ctrl K` (ou `⌘ K`) abre a busca rápida, que leva a qualquer tela, pedido, produto ou ação. `?` mostra todos os atalhos.

Atalho	Ação
`Ctrl K` · `/`	Busca rápida
`G` e depois `V`, `L`, `P`, `R`, `E`, `N`, `U`, `S`	Ir para Visão geral, Loja, Pedidos, Produtos, Estoque, Notas, Usuários ou Sistema
`T`	Alternar tema claro e escuro
`P`	Trocar a paleta de cores
`?`	Lista de atalhos
`Esc`	Fechar janelas e gavetas

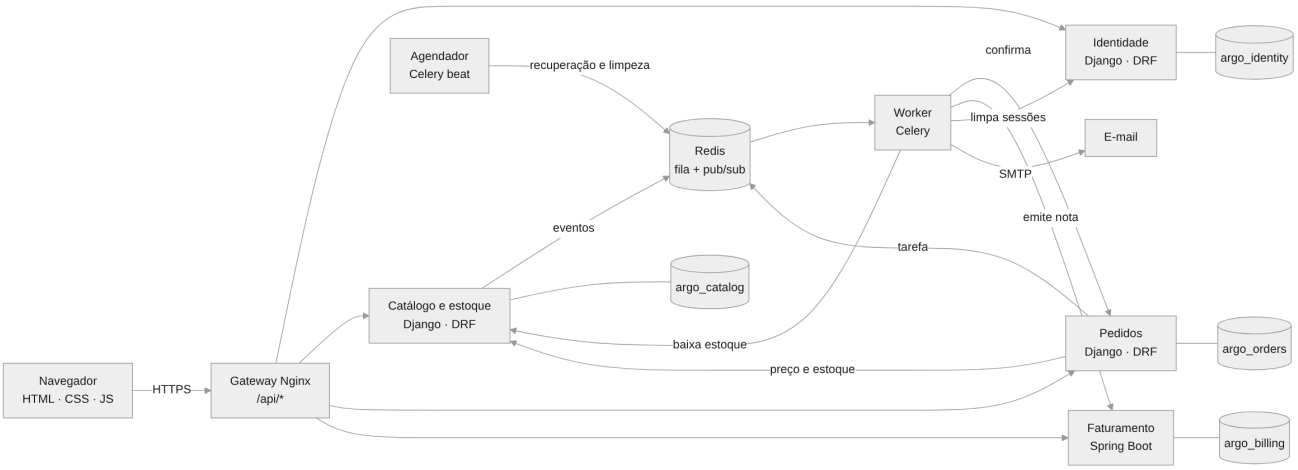


Busca rápida

Capítulo 5

Arquitetura

Cada serviço tem o próprio banco dentro do mesmo MySQL e nunca lê o banco de outro. O que um serviço precisa de outro chega por HTTP com `X-Internal-Token` ou pela fila.



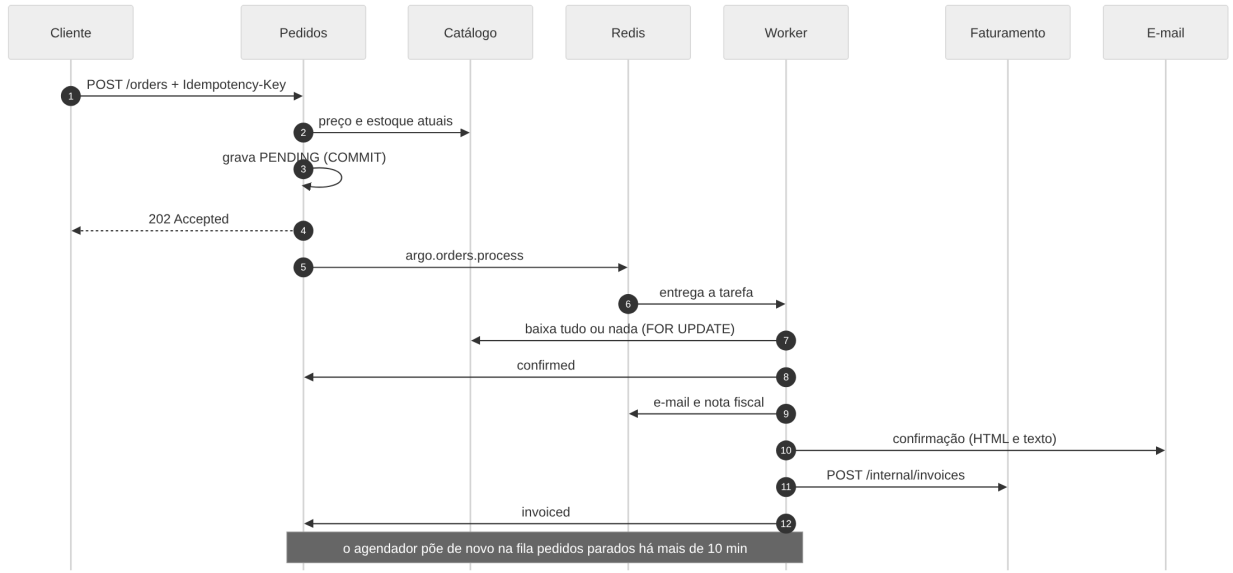
Serviços, filas e bancos

Como o Django está organizado

Os três serviços Python são um único projeto Django (`backend/`). A variável `ARGO_SERVICE` decide qual serviço o processo vira: ele instala só o app daquele serviço, liga só o banco dele e responde só em `/api/<serviço>/` .

Parte	Onde	O que faz
Configuração	`argo/settings.py`	Tudo vem das variáveis `ARGO_*`
Rastreamento	`argo/common/context.py` · `logs.py`	`X-Request-Id` gerado no gateway segue por HTTP e pela fila e aparece em todos os logs (texto ou JSON)
Verificações	`argo/common/checks.py`	`manage.py check --deploy` e o `doctor` apontam segredos de exemplo, curtos ou repetidos e fila em linha em produção
Exportação	`argo/common/export.py`	CSV com BOM, `;` e vírgula decimal, aberto direto no Excel
Comandos	`argo/common/management/commands/`	`serve` , `seed` , `doctor` , `create_admin` , `recover_orders` e `cleanup_tokens`
Tempo real	`argo/catalog/views.py`	`GET /stock/stream` repassa o Pub/Sub do Redis por Server-Sent Events (Daphne, ASGI)
Fila	`argo/worker/` · `argo/orders/dispatch.py`	Celery, Celery em linha (`ARGO_INLINE_QUEUE`) ou gravação nos testes

Do clique à nota fiscal



Fluxo do checkout

1. O cliente fecha o carrinho; o serviço de pedidos confere preço e disponibilidade no catálogo e registra a `Idempotency-Key`.
2. O pedido é gravado como Processando e a resposta volta na hora (`202 Accepted`). Repetir a chamada com a mesma chave devolve o mesmo pedido.
3. O worker baixa o estoque de todos os itens de uma vez, com `SELECT ... FOR UPDATE` em ordem fixa. Se faltar algum item, nada é baixado e o pedido é Recusado.
4. Com o estoque reservado, o pedido vira Confirmado, o e-mail sai e o serviço Java emite a nota.
5. Toda tarefa é idempotente e repetida com espera exponencial se outro serviço estiver fora do ar. O agendador retoma o que tiver ficado para trás.

Capítulo 6

Processamento assíncrono

Celery com Redis cuida de tudo que não precisa travar a resposta ao cliente.

Tarefa	Quando roda	O que faz
`process_order`	Depois do COMMIT do checkout	Baixa o estoque, confirma ou recusa, dispara e-mail e nota
`send_confirmation_email`	Pedido confirmado	E-mail de confirmação com os itens
`send_status_email`	Enviado, entregue ou cancelado	E-mail com a nova situação e o rastreio
`issue_invoice`	Pedido confirmado	Pede a nota ao faturamento em Java
`cancel_order`	Cancelamento	Devolve o estoque com movimentos RELEASE e cancela a nota
`recover_orders`	A cada 5 minutos (agendador)	Retoma pedidos parados há mais de `ARGO_RECOVER_AFTER_MINUTES` e confirmados sem e-mail ou nota
`cleanup_tokens`	Uma vez por dia (agendador)	Apaga refresh tokens vencidos há mais de `ARGO_TOKEN_RETENTION_DAYS` dias

O Flower (http://localhost:5555) mostra filas, workers e falhas. O `X-Request-Id` do checkout acompanha cada tarefa nos logs, então dá para seguir um pedido do clique à nota com um `grep`.

Se o Redis cair, o checkout continua respondendo `202` e o pedido fica em Processando. Quando o Redis volta, o agendador põe o pedido de novo na fila sem intervenção.

Capítulo 7

API

Todas as rotas passam pelo gateway em `/api/<serviço>`, respondem JSON e exigem `Authorization: Bearer <token>`, exceto login, cadastro, refresh e saúde. A documentação interativa fica em `/docs/`.

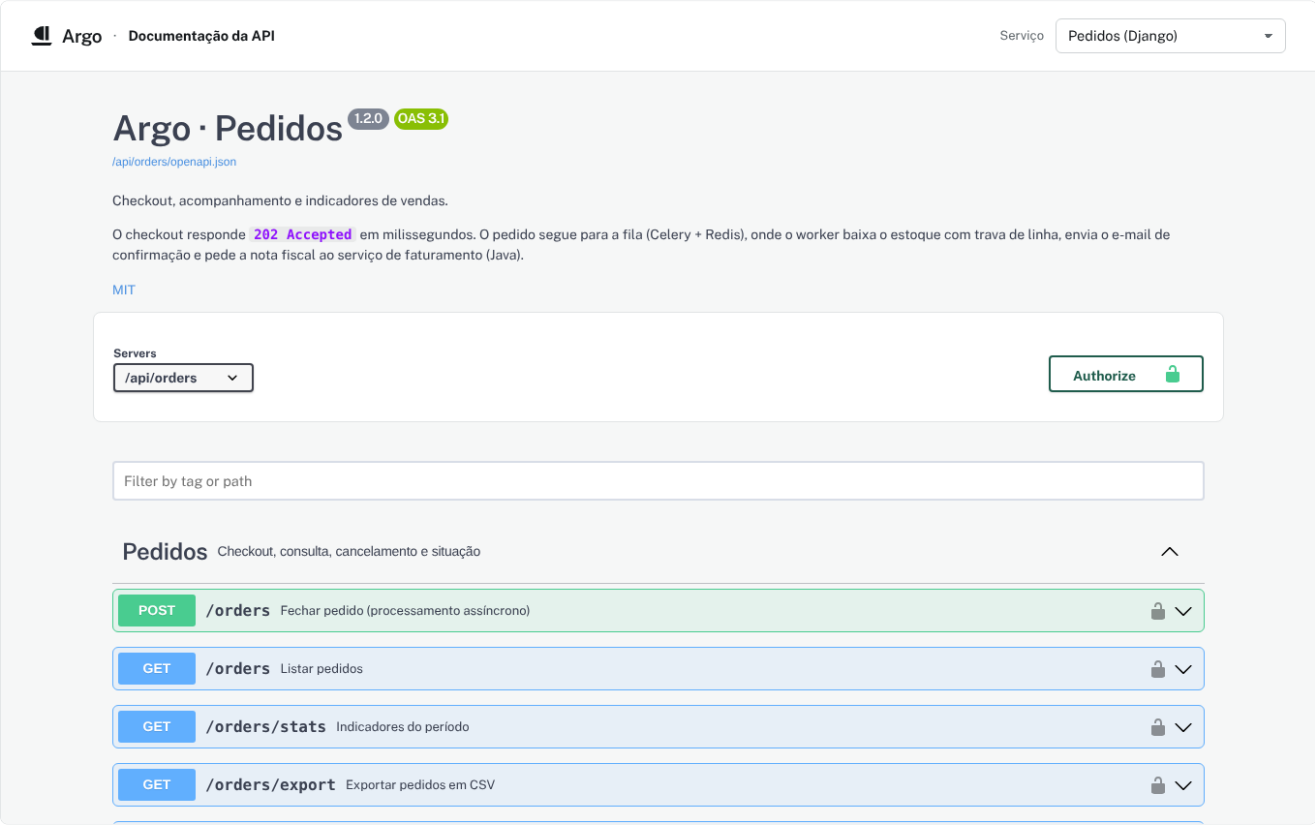
Serviço	Rotas principais
Identidade	`POST /auth/register` · `POST /auth/login` · `POST /auth/refresh` · `POST /auth/logout` · `GET` e `PATCH /auth/me` · `PUT /auth/me/password` · `GET /auth/sessions` · `DELETE /auth/sessions/{id}` · `POST /auth/sessions/revoke-others` · `GET`, `POST` e `PATCH /users` · `POST /users/{id}/password` · `POST /users/{id}/unlock`
Catálogo	`GET /products` (busca, categoria, vendedor, estoque baixo, ordenação) · `GET /products/export` · `GET /products/categories` · `GET`, `POST`, `PUT` e `DELETE /products/{id}` · `POST /products/{id}/stock` · `GET /products/{id}/movements` · `GET /stock/movements` · `GET /stock/summary` · `GET /stock/stream`
Pedidos	`POST /orders` (com `Idempotency-Key`) · `GET /orders` · `GET /orders/export` · `GET /orders/stats` · `GET /orders/{id}` · `POST /orders/{id}/cancel` · `PATCH /orders/{id}/status`
Faturamento	`GET /invoices` (situação, busca e período) · `GET /invoices/export` · `GET /invoices/summary` · `GET /invoices/{número}`
Todos	`GET /health` com a situação do banco (503 quando degradado)

Erros seguem sempre o mesmo formato nos quatro serviços. Toda resposta traz o cabeçalho `X-Request-ID`, que acha a chamada nos logs:

```
{ "code": "validation_error", "message": "Confira os campos.",
  "fields": { "email": "Informe um e-mail válido." } }
```

```
TOKEN=$(curl -s -X POST localhost:8080/api/identity/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"cliente@argo.dev","password":"Argo@2026"}' | jq -r .access_token)

curl -s localhost:8080/api/orders/orders/export?status=CONFIRMED \
  -H "Authorization: Bearer $TOKEN" -o pedidos.csv
```



Documentação da API

Capítulo 8

Configuração

Todas as variáveis ficam no arquivo `.env` (modelo em `.env.example`).

Variável	Padrão	Para que serve
`ARGO_PORT`	`8080`	Porta da aplicação
`ARGO_PUBLIC_URL`	`http://localhost:8080`	Endereço usado nos links dos e-mails
`ARGO_DB_USER` · `ARGO_DB_PASSWORD` · `ARGO_DB_PORT`	`argo` · gerada · `3306`	Usuário e porta do MySQL
`ARGO_JWT_SECRET`	gerado	Assina os tokens (o mesmo para Django e Java)
`ARGO_INTERNAL_TOKEN`	gerado	Segredo das chamadas entre serviços
`ARGO_SEED`	`true`	Cadastra os dados de demonstração em banco vazio
`ARGO_ADMIN_EMAIL` · `ARGO_ADMIN_PASSWORD` · `ARGO_ADMIN_NAME`	vazio	Primeiro administrador, criado na subida
`ARGO_BILLING_TAX_RATE` · `ARGO_BILLING_INVOICE_PREFIX`	`0.1645` · `NF`	Alíquota aproximada e prefixo das notas
`ARGO_SMTP_HOST` · `ARGO_SMTP_PORT`	`mailpit` · `1025`	Servidor de e-mail
`ARGO_SMTP_USER` · `ARGO_SMTP_PASSWORD` · `ARGO_SMTP_TLS`	vazio · `false`	Credenciais do SMTP
`ARGO_MAIL_FROM`	`Argo <nao-responda@argo.dev>`	Remetente dos e-mails
`ARGO_LOG_FORMAT` · `ARGO_LOG_LEVEL`	`text` · `INFO`	`json` para mandar os logs a um agregador
`ARGO_RECOVER_AFTER_MINUTES`	`10`	Tempo até um pedido parado voltar para a fila
`ARGO_TOKEN_RETENTION_DAYS`	`30`	Quanto tempo guardar refresh tokens vencidos
`ARGO_MAILPIT_PORT` · `ARGO_FLOWER_PORT`	`8025` · `5555`	Portas das ferramentas de apoio

Depois de mudar o `.env`, rode `make reiniciar` (ou `argo reiniciar`). `make diagnostico` confere se a configuração nova está funcionando.

Capítulo 9

Testes e automação

Os testes rodam no computador e a cada envio no GitHub Actions.

Suíte	Comando	O que cobre
Python (97 testes, cobertura acima de 90%)	`make testes` ou `cd backend && python -m pytest`	Tokens, papéis, sessões, bloqueio, rotação, permissões, estoque concorrente, idempotência do checkout, filas, recuperação, exportações, e-mails e o fluxo inteiro até a nota
Java (16 testes)	`make testes-java` ou `cd billing && mvn -B verify`	Emissão idempotente, tributos, cancelamento, filtros, exportação CSV, visibilidade por papel, erros, tokens e saúde
Estilo	`make lint`	ruff (formatação e regras) no Python

Os testes Python usam SQLite por padrão. Para rodar no MySQL, defina `ARGO_DATABASE_URL`, `ARGO_CATALOG_DATABASE_URL` e `ARGO_ORDERS_DATABASE_URL` antes do `pytest`.

Integração contínua

- **lint:** ruff check e ruff format.
- **python:** testes no SQLite e num MySQL 8.4, com cobertura mínima de 80% e `check --deploy` com segredos de produção.
- **java:** `mvn verify` com Java 21.
- **docker:** monta as imagens de todos os serviços.
- **Dependabot** abre atualizações semanais de pip, Maven, Docker e Actions; o `.pre-commit-config.yaml` roda o ruff antes de cada commit.

Capítulo 10

Operação e manutenção

Rotinas para manter a plataforma saudável.

Diagnóstico

`make diagnostico` roda `python manage.py doctor` em cada serviço e mostra uma linha por item: banco, migrações pendentes, Redis, SMTP, faturamento e segredos. Termina com erro se algo estiver fora, então serve também em scripts de monitoramento.

Backup

```
make backup
make restaurar ARQUIVO=backups/argo-AAAAMMDD-HHMMSS.sql.gz
```

Os backups usam `mysqldump --single-transaction` dos quatro bancos, sem parar a aplicação, e ficam em `backups/` (fora do Git).

Atualização

`make atualizar` faz `git pull`, um backup e sobe de novo com as imagens novas. As migrações rodam sozinhas e os dados são mantidos.

Logs

Todos os contêineres têm logs com rotação (10 MB × 3). Com `ARGO_LOG_FORMAT=json`, cada linha traz serviço, nível, `request_id` e mensagem; o gateway também grava em JSON com o tempo de resposta de cada chamada.

Administrador de emergência

```
make admin EMAIL=ti@empresa.com.br SENHA='Uma-senha-forte-2026'
argo admin ti@empresa.com.br
```

Capítulo 11

Solução de problemas

Sintoma	O que fazer
Página não abre em localhost:8080	`make situacao` para ver os contêineres; `make logs` mostra o motivo. Confira se a porta `ARGO_PORT` está livre
Pedido parado em Processando	Veja o worker no Flower. O agendador retoma sozinho em até 5 minutos; `make recuperar` força na hora
E-mails não chegam	Com o Mailpit, abra http://localhost:8025. Com SMTP real, `make diagnostico` testa a conexão
Nota não emitida	A tela Sistema mostra se o faturamento está no ar. Quando voltar, a recuperação emite as notas pendentes
Muitas tentativas (429)	Aguarde um minuto. Os limites ficam em `gateway/nginx.conf`
Conta bloqueada	Espere 5 minutos ou peça ao administrador para desbloquear em Usuários
Serviço com problema (503)	O `/health` aponta o banco fora do ar. Confira o MySQL com `make diagnostico`
Aviso de segredo no log	Os segredos ainda são os de exemplo ou curtos. Gere novos no `.env` e rode `make reiniciar`
Tela com cores estranhas	Escolha outra paleta em Minha conta ou aperte `P`; a escolha fica só no navegador

Para qualquer erro da API, copie o cabeçalho `X-Request-ID` da resposta: `make logs | grep <valor>` mostra o caminho completo da chamada, do gateway ao worker.

Capítulo 12

Novidades da 1.2.0

Mais recursos de uso, configuração e manutenção, mantendo o visual.

Área	O que mudou
Uso	Busca rápida (`Ctrl K`) e atalhos de teclado; tema claro, escuro ou do sistema; cinco paletas de cores; Minha conta com nome, senha e sessões; página do produto; filtros e exportação CSV em pedidos, produtos e notas; tela Sistema; feed de estoque com filtro; comparação com o período anterior no painel; impressão de pedidos e notas; instalação como aplicativo (manifest)
Pedidos	Checkout com `Idempotency-Key` ; e-mail ao cliente quando o pedido é enviado, entregue ou cancelado; recuperação automática de pedidos parados
Contas	Sessões com aparelho e IP, encerramento remoto, troca de senha sem derrubar a sessão atual, redefinição e desbloqueio pelo administrador, primeiro administrador por variável de ambiente
Operação	`Makefile` e `argo` para Windows; `doctor` , `create_admin` , `recover_orders` e `cleanup_tokens` ; agendador Celery beat; backup e restauração; `/health` com checagem do banco; logs em JSON com `X-Request-Id` ponta a ponta; rotação de logs
Segurança	Content-Security-Policy; limite geral de requisições na API; aviso de segredos de exemplo ou curtos; `X-Request-ID` em todas as respostas
Qualidade	97 testes Python e 16 Java; job de lint no CI; Dependabot; pre-commit

Como atualizar

Para atualizar uma instalação 1.1.0: `make atualizar` (ou `git pull` e `docker compose up -d --build`). As migrações `identity.0002_sessoes` e `orders.0003_chave_de_idempotencia` rodam sozinhas e os dados são mantidos. O novo contêiner `scheduler` sobe junto.

Versão 1.1.0

Correções no gateway e nos totais em BIGINT, checkout resiliente com o Redis fora do ar, rotas internas bloqueadas no gateway, limite de tentativas no login, gzip e cache dos arquivos estáticos, `GET /healthz` e erros 429, 502, 503 e 504 em JSON.